Coding e Fondamenti di Intelligenza Artificiale

A.A. 2026/27 — Lezione 11: Software 2.0

Vincenzo Nardelli



vincenzo.nardelli@unicatt.it · vincnardelli.com/cia

LAB Dall'autovelox scritto all'autovelox appreso

Nel modulo 2 abbiamo scritto a mano la regola dell'autovelox:
0 · 36 · 148 · 370 · 500 €, con soglie a 0, 10, 40 e 60 km/h oltre il limite.

Oggi facciamo il contrario: diamo a un algoritmo
5.000 multe già emesse
e gli chiediamo di trovare le soglie da solo.

È il primo modello di **machine learning** del corso,
ed è anche il primo codice in **Python**.

LAB Software 1.0 e Software 2.0

Scritto a mano

regola + dati → risposte

- una persona legge la legge e scrive gli `if`
- deterministico: stessa auto, stessa multa
- se la legge cambia, si riscrive il codice

Imparato dai dati

dati + risposte → regola

- l'algoritmo guarda esempi già classificati
- trova le soglie da solo
- è bravo quanto gli esempi che ha visto

Un modello di machine learning è **una funzione che nessuno ha scritto**.

LAB Google Colab

1. Dal sito: `vincnardelli.com/cia` → modulo 8 → **Apri in Colab**
2. Il notebook è fatto di celle di testo e celle di codice
3. **Shift + Invio** esegue una cella; il risultato compare sotto
4. Le celle si eseguono dall'alto in basso. Per ripartire: *Runtime* → *Esegui tutto*
5. Per tenere le modifiche: *File* → *Salva una copia in Drive*

Niente da installare: il codice gira sui computer di Google.
I dati si leggono direttamente dal sito del corso.

LAB Da R a Python

R	Python	
`x <- 5`	`x = 5`	assegnare
`c(120, 80, 45)`	`[120, 80, 45]`	una lista
`x[1]`	`x[0]`	si conta da 0
`length(x)`	`len(x)`	
`function(a) { }`	`def f(a):` + rientro	niente graffe
`else if`	`elif`	
`for (i in 1:n)`	`for i in range(n):`	da 0 a n–1
`library(dplyr)`	`import pandas as pd`	
`read.csv("f.csv")`	`pd.read_csv("f.csv")`	
`df$x`	`df["x"]`	
`filter(df, x > 5)`	`df[df["x"] > 5]`	
`count(x)`	`df["x"].value_counts()`	
`group_by(g) %>% summarise(mean(x))`	`df.groupby("g")["x"].mean()`	

LAB L'autovelox appreso - La multa scritta a mano

```
def multa(velocita, limite):
    differenza = velocita - limite
    if differenza <= 0:
        return 0
    elif differenza <= 10:
        return 36
    elif differenza <= 40:
        return 148
    elif differenza <= 60:
        return 370
    else:
        return 500
```

Lo stesso `if/else` del modulo 2: le graffe diventano **rientro** di quattro spazi, `else if` diventa `elif`.

`multa(95, 90)`	36
`multa(100, 90)`	36
`multa(101, 90)`	148
`multa(160, 90)`	500

Applicata alle 5.000 auto del file, dà la stessa multa in tutte.

LAB L'autovelox appreso - L'albero impara

```
from sklearn.tree import DecisionTreeClassifier, export_text

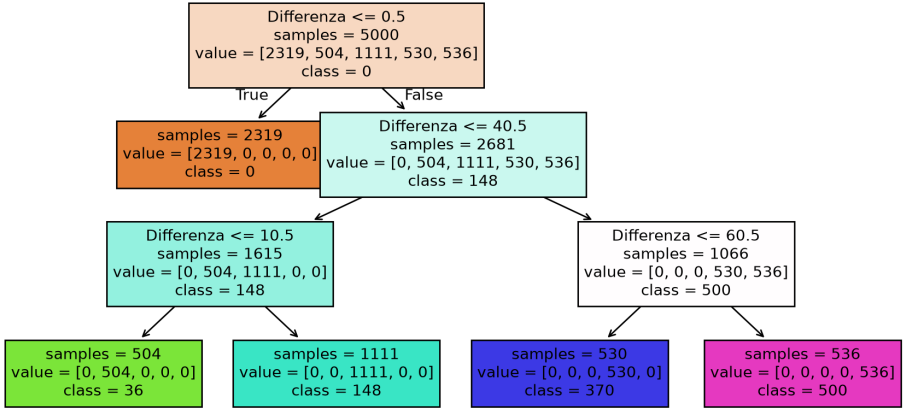
X = autovelox[["Differenza"]] # quello che sappiamo di ogni auto
y = autovelox["Multa"] # quello che vogliamo prevedere

albero = DecisionTreeClassifier(random_state=1)
albero.fit(X, y) # impara dagli esempi
albero.predict(nuove_auto) # risponde su auto nuove
```

Un **albero di classificazione** prova tutte le soglie possibili sulla differenza, sceglie quella che separa meglio le multe, e ripete su ognuno dei due gruppi.

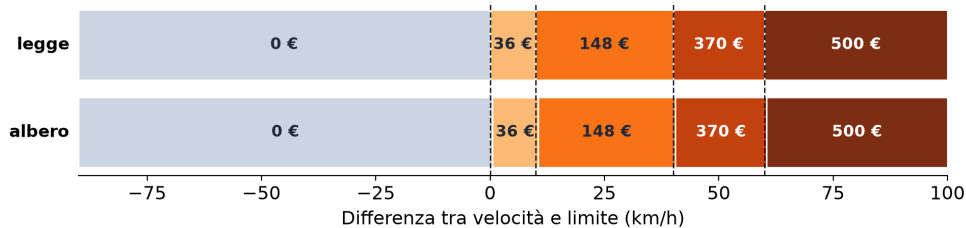
In scikit-learn ogni modello si usa così: si crea, si allena con `fit`, si usa con `predict`.

LAB L'autovelox appreso - L'albero



Le soglie sono **0,5 · 10,5 · 40,5 · 60,5**: sugli interi, "≤ 10,5" vuol dire "≤ 10".

LAB L'autovelox appreso - Le soglie trovate



Sono le soglie dell'articolo 142. Nessuno gliele ha dette: le ha trovate guardando gli esempi.
Sulle 5.000 auto l'albero dà la stessa multa della legge nel **100%** dei casi.

LAB L'autovelox appreso - Cosa abbiamo imparato

1. Un albero di classificazione trova da solo le soglie di una regola, guardando esempi già classificati.
2. In scikit-learn ogni modello si usa così: si crea, si allena con `fit`, si usa con `predict`.

Ma il 100% è sulle stesse multe da cui l'albero ha imparato.
Ha imparato davvero? Lo vediamo nella prossima lezione: training e test.

Esplorazione sul sito: **Scritto o imparato?** — guardate l'albero crescere un nodo alla volta.